gram.en: Comprehensive Notes on Explainable Grammatical Analysis

Sinan Olsson-Pasic

May 31, 2026

Abstract

gram.en is an explainable grammatical analysis engine. It combines finite-state morphology, Earley parsing, feature-structure unification, and targeted error rules in order to diagnose not only whether an input sentence is grammatical, but which grammatical constraint failed and where. These notes develop the theory and the implementation together. The mathematical thread explains why the engine uses finite automata for morphology, a context-free backbone for syntax, and feature unification for local constraints. The engineering thread shows how those ideas become a compiler-like pipeline for natural language.

Contents

1 Introduction

This paper documents the design and development of **gram.en**, an explainable grammatical analysis engine built from ideas in formal language theory, complexity theory, compiler design, and computational linguistics. The engine analyzes natural-language input through a compiler-like pipeline: tokenization, finite-state morphology, lexical lookup, parsing, feature-structure unification, and rule-level diagnostic reporting.

The motivating question was simple: *Can a natural language be represented as a set of executable rules?* Programming languages such as C, Python, and R are intentionally formal systems. Their syntax and behavior is specified, and compilers can reject invalid input according to fixed rules. Natural languages such as English, Swedish, and Russian are different. They are ambiguous, exception-heavy, historically layered, and only partially captured by explicit rules. Human speakers nevertheless internalize many of these patterns through exposure: we often judge whether a sentence "sounds right" without consciously naming the rule being applied.

gram.en attempts to make part of that implicit knowledge explicit. It treats grammatical analysis as a static-analysis problem over natural language. A sentence is tokenized, morphologically analyzed, parsed against a context-free backbone, and checked by unifying grammatical feature structures such as number, person, case, tense, and verb form. When a constraint fails, the engine attempts to report not only that the sentence is ungrammatical, but which rule failed, where it failed, and what repair follows from that failure.

This is the central design claim of **gram.en**: grammatical verdicts and grammatical explanations should come from the same computation. The system should not first reject a sentence and then search for an unrelated explanation after the fact. Instead, the failed parse, failed unification, or matched mal-rule should itself contain the information needed for the report.

The goal is not to replace modern statistical or neural grammar-correction systems. Those systems are far better suited to broad coverage, graded acceptability, and idiomatic language use. **gram.en** is instead a proof of concept for a different trade-off: limited coverage in exchange for explicit structure, predictable behavior, and explainable diagnostics. The central design claim is that grammatical verdicts and grammatical explanations should come from the same computation.

2 Languages as sets of strings

A grammar engine needs a precise meaning for the statement "this sentence is valid." In ordinary speech, a language is a social and historical object. For a computer scientist, the first abstraction is simpler: a language is a set of strings. If Σ is an alphabet, then Σ^* is the set of all finite strings over that alphabet, and a language is any subset $L \subseteq \Sigma^*$.

This abstraction is deliberately austere. It ignores meaning, speakers, context, and style. But it gives the engine a clear decision problem: given an input string w, decide whether $w \in L$. A grammar is a finite description of such a set.

Definition 2.1 (Generative grammar)**.** A generative grammar is a tuple

$$G = (V_N, V_T, R, S),$$

where V_N is a finite set of nonterminals, V_T is a finite set of terminals with $V_N \cap V_T = \emptyset$, R is a

finite set of rewrite rules, and $S \in V_N$ is the start symbol. A rule is a pair (P, Q) with

$$P \in (V_N \cup V_T)^+, \qquad Q \in (V_N \cup V_T)^*.$$

A rule (P, Q) licenses one rewriting step. If P appears inside a larger string $\alpha P \beta$, then the rule rewrites that string as $\alpha Q \beta$. Write this one-step relation as $\Rightarrow$.

Definition 2.2 (Derivation)**.** The reflexive–transitive closure of $\Rightarrow$ is written $\Rightarrow^*$. Thus $\alpha \Rightarrow^* \gamma$ means that γ can be obtained from α by zero or more rule applications.

The language generated by G is then

$$L(G) = \{\, w \in {V_T}^* \mid S \Rightarrow^* w \,\}.$$

A derivation is a proof of membership. If $S \Rightarrow^* w$, then the grammar contains a witness that w belongs to the language. If no derivation exists, then w lies outside the language described by this particular grammar. That last phrase matters: outside $L(G)$ does not necessarily mean outside English. It may only mean outside the fragment that has been implemented.

2.1 Finite descriptions and infinite languages

Grammars are finite objects, but the languages they describe may be infinite. The grammar for balanced parentheses does not list every balanced string; it gives a recursive pattern that generates all of them. This is the first reason formal grammar is useful: finite rules can describe infinite regularities.

There is also a boundary. Not every language over an alphabet can be described by a finite grammar or program. There are only countably many finite strings that could serve as descriptions, but uncountably many subsets of Σ^*. Therefore most languages are not finitely describable at all. For this engine, the consequence is practical rather than philosophical: the goal is not to describe "all of English." The goal is to choose a useful, computable fragment whose rules can be enforced and explained.

3 The computational cost of grammar

A grammar formalism is not just a linguistic choice, but rather serves as an algorithmic budget. Once the engine chooses a grammar class, it also chooses what kind of recognizer it can use and how expensive membership will be.

The Chomsky hierarchy organizes grammars by the shape of their rewrite rules. Restricting rule shape restricts memory, and restricting memory changes what dependencies can be recognized.

Definition 3.1 (The four grammar types)**.** A grammar (V_N, V_T, R, S) is:

1. **Type 0**, or recursively enumerable, if there is no restriction on the rules.
2. **Type 1**, or context-sensitive, if rules are non-contracting: $|P| \leq |Q|$.
3. **Type 2**, or context-free, if every left-hand side is a single nonterminal: $A \to Q$.
4. **Type 3**, or regular, if every rule is right-linear, for example $A \to aB$ or $A \to a$.

Type	Class	Recognizer	Memory resource
0	recursively enumerable	Turing machine	unbounded tape
1	context-sensitive	linear-bounded automaton	tape linear in input
2	context-free	pushdown automaton	one stack
3	regular	finite automaton	finite control only

Table 1: Grammar power tracks memory.

The inclusions are strict:

$$\textsc{Reg} \subsetneq \text{CF} \subsetneq \text{CS} \subsetneq \text{RE}.$$

Regular languages are recognized with no unbounded memory. Context-free languages add one stack, enough to match nested dependencies. Context-sensitive languages allow more general bounded work over the input. Recursively enumerable languages allow arbitrary computation.

3.1 Membership as the engine's core decision problem

The direct question asked by a recognizer is membership: given a grammar G and input w, is $w \in L(G)$? The answer becomes more expensive as the grammar formalism becomes more powerful.

Theorem 3.1 (Cost of membership)**.** *For a fixed grammar and input length n, regular membership is $O(n)$, context-free membership is decidable in $O(n^3)$ by standard dynamic programming methods such as CKY, context-sensitive membership is PSPACE-complete, and unrestricted Type 0 membership is undecidable.*

Proof sketch. A deterministic finite automaton reads the input once, so regular recognition is linear. For context-free grammars in Chomsky normal form, CKY fills a triangular table of spans; there are $O(n^2)$ spans and $O(n)$ split points per span, giving $O(n^3)$ recognition for a fixed grammar. A linear-bounded automaton has only polynomially many configurations but may need polynomial space to search them, placing context-sensitive membership in PSPACE, with a matching lower bound. For unrestricted grammars, derivations can simulate Turing machines, so deciding membership would decide the halting problem. □

This theorem is one of the engine's design constraints. A fully general grammar is not merely slow; it is unanswerable. A practical grammar checker therefore has to live near the context-free level: expressive enough to build phrase structure, but restricted enough to keep recognition decidable and polynomial.

Definition 3.2 (Weak and strong generative capacity)**.** The *weak* generative capacity of a grammar is the string set $L(G)$. The *strong* generative capacity is the set of structures, such as parse trees, that the grammar assigns. Two grammars can be weakly equivalent while assigning different structures.

A yes/no grammar checker asks a weak-capacity question. An explainable grammar checker asks a strong-capacity question. To say that `the dog bark` violates subject–verb agreement, the engine needs more than rejection; it needs the subject, the verb phrase, the rule that combined them, and the feature values that failed to unify.

4 Pumping lemmas: proving that memory is necessary

The hierarchy says that machines with more memory can recognize more languages. But to show that the inclusions are genuinely strict, we need negative arguments: proofs that a language cannot be recognized by a weaker machine. Pumping lemmas provide those tests.

A pumping lemma says that every sufficiently long string in a language from a given class contains a repeatable part. For regular languages, the repeatable part comes from a finite automaton revisiting a state. For context-free languages, it comes from a parse tree revisiting a nonterminal along a root-to-leaf path. In both cases the machine has used some bounded resource twice, so some corresponding part of the string can be duplicated or deleted while staying in the language.

The usual proof strategy is by contradiction. Assume the language belongs to a class. Then every long enough string in it must be pumpable. Choose a string whose dependency structure cannot survive pumping. If every legal pumping decomposition breaks the language, the language could not have belonged to the class.

4.1 The regular pumping lemma

Lemma 4.1 (Regular pumping)**.** *If L is regular, then there is a pumping length $p \geq 1$ such that every string $s \in L$ with $|s| \geq p$ can be written $s = xyz$, where $|xy| \leq p$, $|y| \geq 1$, and $xy^i z \in L$ for every $i \geq 0$.*

Proof sketch. Let p be the number of states in a deterministic finite automaton for L. Any accepting run over a string of length at least p must revisit a state among the first p input positions. The substring read between the two visits is a loop, and the automaton can traverse that loop any number of times while still accepting. □

Proposition 4.2. *The language $\{a^n b^n \mid n \geq 0\}$ is not regular.*

Proof. Assume it is regular and let p be its pumping length. Choose $s = a^p b^p$. Since $|xy| \leq p$, the pumpable substring y contains only a's. Pumping it up gives xy^2z, which has more a's than b's and is therefore not in the language, contradicting the pumping lemma. □

The point is not the letters a and b. The point is the dependency shape. The language $\{a^n b^n\}$ requires matching two unbounded counts. A finite automaton has no stack on which to remember how many a's it saw. Center embedding in natural language has this nested shape: "the rat [the cat [the dog chased] killed] ate" contains dependencies that must be closed in reverse order. This is why syntax cannot be purely finite-state.

4.2 The context-free pumping lemma

One stack can handle nesting, but it cannot handle every kind of dependency. In particular, it cannot generally compare three independent counts or copy an arbitrary string.

Lemma 4.3 (Context-free pumping)**.** *If L is context-free, then there is a pumping length $p \geq 1$ such that every string $z \in L$ with $|z| \geq p$ can be written $z = uvwxy$, where $|vwx| \leq p$, $|vx| \geq 1$, and $uv^i wx^i y \in L$ for every $i \geq 0$.*

Proof sketch. In a sufficiently tall parse tree, some nonterminal must repeat along a path from the root to a leaf. The portion of the tree between the two occurrences can be duplicated or deleted, which pumps two substrings, v and x, together. □

Proposition 4.4. *The language* $\{a^n b^n c^n \mid n \geq 0\}$ *is not context-free.*

Proof sketch. Choose $a^p b^p c^p$. The substring vwx has length at most p, so it can span at most two of the three blocks. Pumping changes at most two of the three counts, leaving the equality of all three counts broken. □

This matters because some natural-language dependencies are not purely nested. Shieber's Swiss German argument uses cross-serial dependencies: a sequence of noun phrases is followed by a sequence of verbs, and each verb is linked to the corresponding noun phrase rather than to the most recently opened one.

Theorem 4.5 (Shieber 1985, informal)**.** *Some natural-language string sets are not context-free.*

Proof architecture. Context-free languages are closed under intersection with regular languages and under homomorphism. Shieber isolates a Swiss German fragment D such that, for a suitable regular language R and homomorphism h,

$$h(D \cap R) = \{\, a^m b^n c^m d^n \mid m, n \geq 0 \,\}.$$

The crossed agreement dependencies force the matched counts. Since the resulting language is not context-free, and context-free languages are closed under the operations used to obtain it, D cannot have been context-free. □

The engineering consequence is conservative. Natural language is not strictly context-free, but the constructions that witness this are not the primary source of everyday grammar-checking errors. **gram.en** therefore uses a context-free backbone plus feature constraints. It deliberately defers mildly context-sensitive phenomena in exchange for a simpler, polynomial, explainable core.

Definition 4.1 (Mild context-sensitivity)**.** A class of languages is mildly context-sensitive if it properly contains the context-free languages, captures limited cross-serial dependencies, has semilinear growth, and remains parsable in polynomial time.

Tree-Adjoining Grammar, Combinatory Categorial Grammar, Linear Indexed Grammar, and Head Grammar are weakly equivalent in the sense of Vijay-Shanker and Weir. This is a useful landmark, but not the target of the current engine. The current engine chooses a practical point below that line: finite-state morphology, context-free phrase structure, and unification constraints.

5 From generation to constraints

The grammars above are generative: start from S and rewrite until a terminal string appears. For diagnosis, it is often more useful to invert the viewpoint. A grammar can be treated as a set of constraints that a candidate structure must satisfy. Grammaticality becomes the existence of a structure for the input whose constraints are all consistent.

This is the step that turns recognition into explanation. The same constraint system that accepts a well-formed structure also identifies the point of failure in an ill-formed one. A failed constraint

is local: it mentions particular constituents. It is also named: it is a particular agreement, case, or selection rule. That gives the engine something to report.

5.1 Feature structures

Natural-language rules often talk about properties rather than just categories. A noun phrase may be singular or plural; a pronoun may be nominative or accusative; a verb may be finite, base-form, past-tense, or participial. These properties are represented as feature structures.

Definition 5.1 (Feature structure)**.** A feature structure is a finite directed acyclic graph with labelled feature arcs and atomic values at leaves. Equivalently, it is a partial map from feature paths to values, such as

$$[\,\textsc{cat} : \mathrm{N},\ \textsc{num} : \textsc{sg},\ \textsc{pers} : 3,\ \textsc{case} : \textsc{nom}\,].$$

Definition 5.2 (Subsumption and unification)**.** A feature structure F subsumes G, written $F \sqsubseteq G$, when G contains all the information in F and possibly more. The unification $F \sqcup G$ is the least structure that contains the information from both. If the two structures demand incompatible atomic values on the same path, unification fails.

Operationally, unification is the grammar engine's type-checking step. Compatible information merges. Incompatible information produces a local failure.

$$[\textsc{num} : \textsc{sg}] \sqcup [\textsc{num} : \textsc{pl}] = \top.$$

The symbol $\top$ here denotes inconsistency. A subject requiring singular agreement and a verb requiring plural agreement cannot both be true in the same structure.

Theorem 5.1 (Unification as a join)**.** *If inconsistent structures are represented by a top element $\top$, then unification is the least upper bound operation in the subsumption order. It succeeds exactly when the two feature structures have a common refinement.*

Proof sketch. Compatible feature information can be merged path-by-path. If two paths assign the same atomic value, they agree; if one path is absent, the present value is retained; if both assign distinct atomic values, no structure can satisfy both. The successful merge is the least structure that contains both inputs because it adds no information except what one of the inputs already required. □

For the engine, this is the mechanism that makes explanations possible. A parse failure caused by unification is not a bare rejection. It has a rule, two constituents, a feature path, and two incompatible values.

6 Gradience and the coverage trap

The formal model above is crisp: either a string belongs to $L(G)$ or it does not. Natural language is not always crisp. Speakers make graded judgments using labels like acceptable, awkward, questionable, or unacceptable. This is one reason modern grammar correction often uses statistical and neural models: those models are good at broad coverage and graded preferences.

The more dangerous issue for a rule engine is coverage. A hand-written grammar G defines only its implemented fragment. Therefore

$$\neg(w \in L(G)) \neq \text{“}w \text{ is ungrammatical.”}$$

No parse may mean that the sentence is wrong. It may also mean that the grammar has not learned the construction yet. Treating every failed parse as an error manufactures false positives.

gram.en therefore separates three outcomes:

1. A full parse succeeds: the sentence is grammatical within the implemented fragment.
2. A known constraint or mal-rule identifies a violation: the engine reports a localized grammatical error.
3. No parse and no known diagnostic succeeds: the input is out of coverage.

This is a design rule, not just a philosophical caution. The engine should only report an error when it can point to the rule that was violated.

7 The engine as a compiler-like pipeline

The implementation follows the same staged discipline as a compiler front end. Each stage transforms one representation into another and passes ambiguity forward until later constraints can resolve it.

```
text -> tokenizer -> morphology -> lexicon -> parser(+unification)
                    -> error detection -> report
```

Stage	Responsibility
Tokenizer	Converts raw text into tokens, including contractions such as `don't -> do + n't`.
Morphology	Maps surface forms to possible lemmas, categories, and inflectional features.
Lexicon	Attaches categories and feature structures to closed-class words and analyzed stems.
Parser	Builds constituent structure using an Earley chart parser over a context-free backbone.
Unification	Enforces agreement, case, verb-form, and selection constraints.
Diagnostics	Converts localized failures and mal-rule matches into human-readable reports.

Table 2: The analysis pipeline.

The pipeline is language-independent in shape. The language-specific data live in the lexicon, the morphology, and the grammar file. Ambiguity enters early: `bark` may be a noun or a verb, and `barked` may be finite past or a past participle. The parser keeps competing analyses alive and lets structure decide which ones fit.

8 Tokenization

Tokenization maps a character string to a sequence of word-like units. For space-delimited languages this can look trivial, but it is still a linguistic decision. Contractions, clitics, punctuation, hyphenation, capitalization, and numbers all need policy.

In the current English fragment, contraction splitting is essential:

$$\texttt{don't} \mapsto \texttt{do} + \texttt{n't}, \qquad \texttt{I'm} \mapsto \texttt{I} + \texttt{'m}.$$

The clitic itself is not discarded. It becomes a token with lexical and syntactic behavior. This allows the grammar to distinguish `I don't bark`, `he doesn't bark`, and `me'll bark` using the same syntactic machinery that checks agreement and case elsewhere.

For languages without spaces, such as Japanese, tokenization becomes a major analysis problem in its own right. The current engine does not solve that problem; it treats tokenization as a replaceable stage.

9 Finite-state morphology

Morphology is where word forms are decomposed into stems and inflectional information. English has relatively light morphology, but even English needs productive patterns:

$$\texttt{dogs} \mapsto \texttt{dog + N + Pl}, \qquad \texttt{liked} \mapsto \texttt{like + V + Past}, \qquad \texttt{tried} \mapsto \texttt{try + V + Past}.$$

Listing every inflected form by hand works for a toy grammar and collapses immediately when the lexicon grows. The right abstraction is finite-state morphology.

9.1 Finite-state transducers

A finite-state automaton recognizes a language: it accepts or rejects a string. A finite-state transducer recognizes a relation between strings: it reads one string and writes another. An arc is labelled $a : b$, meaning read a on the input tape and write b on the output tape. The empty symbol ε allows insertion or deletion.

Definition 9.1 (Finite-state transducer)**.** A finite-state transducer is a tuple

$$T = (Q, \Sigma, \Gamma, E, q_0, F),$$

where Q is a finite state set, Σ is the input alphabet, Γ is the output alphabet, $q_0 \in Q$ is the start state, $F \subseteq Q$ is the accepting set, and $E \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \times Q$ is the arc relation.

The engine uses transducers in both directions. In generation, a lexical analysis produces a surface word. In analysis, a surface word recovers possible lexical analyses. This symmetry is one reason finite-state methods are standard in morphology.

9.2 Continuation lexicons and tries

The morphology compiler uses a LEXC-like idea: roots point to paradigms, and paradigms list the forms that a root can take.

```
%lex Root
    dog  : N-reg <lemma>=dog
    cat  : N-reg <lemma>=cat
    bark : V-reg <lemma>=bark

%lex N-reg : N
    +N+Sg : 0 <num>=sg <pers>=3
    +N+Pl : s <num>=pl <pers>=3

%lex V-reg : V
    +V+Inf  : 0  <vform>=bare
    +V+Past : ed <vform>=fin <tense>=past
```

A naive implementation builds one path per root-form pair. The current engine instead builds a trie over stems and shares paradigm sub-FSTs. If many words share prefixes, the trie collapses common structure. If many roots share a paradigm, the continuation transducer is built once.

The state-count effect is visible in benchmark data:

stems	states	arcs	build	states/stem
100	514	734	0.5 ms	5.14
1,000	4,542	6,563	3.0 ms	4.54
10,000	38,109	58,130	47 ms	3.81
50,000	164,764	264,785	271 ms	3.30

Table 3: Trie prefix-sharing reduces states per stem as the lexicon grows.

The mathematical relation recognized by the transducer is unchanged. The implementation is smaller because equivalent prefixes and paradigm continuations are shared.

9.3 Replace rules

Concatenating a root and suffix is not always enough. English past tense illustrates the problem:

$$\texttt{like + ed} \mapsto \texttt{liked}, \qquad \texttt{try + ed} \mapsto \texttt{tried}.$$

Morphophonemic spelling rules mediate between an ideal lexical representation and the surface form.

A replace rule has the abstract form

$$A \to B \;|||\; L\_R,$$

read: rewrite A as B in the context where L appears to the left and R to the right. Each rule denotes a regular relation, so a cascade of rules can be represented by composed finite-state transducers.

In the grammar language, character classes make these rules readable:

```
%class Vowel : a | e | i | o | u
%class Cons  : b | c | d | f | g | h | j | k | l | m | n | p | r | s | t | v | w | x | y
    | z
```

```
%rule e:0 => Cons _ e d
%rule y:i => _ e d
```

These rules allow `liked` and `tried` to be analyzed productively instead of being listed as one-off lexical entries.

9.4 Input-ε elimination

A transducer may contain arcs that consume no input. They are mathematically convenient, but operationally dangerous. A depth-first application algorithm can spend exponential time walking many different ε-paths that reach the same state and input position.

The standard fix is input-ε elimination. For every state s, compute its input-ε closure: the states reachable from s using arcs whose input label is ε, together with the output strings accumulated along those paths. For every real input-consuming arc leaving a state in that closure, install an equivalent arc directly from s whose output includes the accumulated closure output.

Formally, if

$$s \xrightarrow{\varepsilon:w} t \quad \text{and} \quad t \xrightarrow{x:y} u,$$

then the rebuilt transducer contains an arc

$$s \xrightarrow{x:wy} u.$$

Residual accepting paths through input-ε arcs are represented as accepting output at the state. Every run of the original machine factors into input-ε segments followed by input-consuming arcs, so every run has a corresponding run in the rebuilt machine, and conversely. The recognized relation is preserved.

For the engine, the payoff is direct: after elimination, every path step that matters consumes input. The search depth is bounded by the length of the word rather than by the number of hidden ε-paths in the graph.

The same machinery extends to adjectives. A regular adjective paradigm derives positive, comparative, and superlative forms:

`small` $\mapsto$ `smaller`, `smallest`, `happy` $\mapsto$ `happier`, `happiest`, `large` $\mapsto$ `larger`, `largest`.

The spelling rules are not adjective-specific hacks. Silent-`e` deletion, `y`$\to$`i`, and consonant doubling are morphophonemic rules in the rewrite cascade. The lexicon then filters the cascade by requiring the derived surface form to correspond to a known root and paradigm.

10 Parsing with an Earley chart

Morphology gives possible word analyses. Syntax must decide how those analyses combine. The engine uses an Earley parser because it can handle arbitrary context-free grammars, preserve ambiguity, and expose a useful chart trace.

An Earley item records a partially recognized production:

$$[A \to \alpha \bullet \beta, i, j],$$

meaning that the parser began recognizing A at position i, has recognized α up to position j, and still expects β.

The algorithm has three operations:

1. **Predict.** If the dot is before a nonterminal B, add items for productions of B.
2. **Scan.** If the dot is before a terminal that matches the next token, advance the dot.
3. **Complete.** If an item finishes a constituent, advance any earlier item that was waiting for that constituent.

In a purely context-free parser, completion only checks category symbols. In **gram.en**, completion also applies feature equations. If the rule's equations unify successfully, the completed constituent carries the merged feature structure. If unification fails, the item is blocked and its failure can become a diagnostic candidate.

11 Feature equations in grammar rules

A phrase-structure rule builds categories. A feature equation states what must be true about their grammatical properties.

```
S -> NP VP
    <NP agr> = <VP agr> ! "subject-verb agreement" fix: agree-verb

NP -> Det N
    <NP agr> = <N agr>

VP -> V
    <VP agr> = <V agr>
```

The context-free backbone says that a sentence can consist of an NP followed by a VP. The feature equation says that this is only a valid sentence if their agreement features are compatible. Thus the parser and the constraint checker are not separate systems; constraint checking happens while parse structure is built.

There is one important engineering caveat. If a production contains the same category twice, paths such as `<NP num>` are ambiguous. Coordination and ditransitives make this unavoidable:

```
NP -> NP Conj NP
VP -> V NP NP
```

The principled fix is indexed or aliased constituent addressing, such as `<NP.1 num>` and `<NP.2 num>`, or named production positions such as `left:NP` and `right:NP`. This is best treated as an architectural requirement before large-scale rule expansion.

12 Diagnostics: from failure to explanation

The coverage trap says that no parse is not enough. The engine must only report errors it can localize. **gram.en** uses two diagnostic mechanisms.

12.1 Constraint relaxation

If strict parsing fails, the parser can retry while relaxing one tagged equation. If relaxing exactly that equation allows a full parse, then the equation is a plausible diagnosis.

For example, the rule

```
S -> NP VP
    <NP agr> = <VP agr> ! "subject-verb agreement" fix: agree-verb
```

names the agreement constraint. If `the dog bark` fails strictly but succeeds when this constraint is relaxed, the engine reports subject–verb agreement rather than a generic parse failure.

12.2 Mal-rules

Some mistakes are better recognized directly. A mal-rule is an intentionally wrong pattern with an attached diagnostic and repair.

```
%mal S -> NP VP
    *ERR "subject-verb agreement: missing third-person singular -s"
    *FIX "inflect the verb or change the subject number"
```

Mal-rules are precise but anticipatory: the grammar author has to know the error pattern in advance. Constraint relaxation is more general but can over-trigger when many repairs are possible. A practical engine uses both and ranks minimal, localized explanations above broad ones.

13 Worked example: `the dog bark`

Consider the input:

`the dog bark`

The tokenizer returns three tokens. The lexicon and morphology produce:

$$\begin{aligned} \texttt{the} &\mapsto \mathsf{Det}, \\ \texttt{dog} &\mapsto \mathsf{N}[\mathsf{num} = \mathsf{sg},\ \mathsf{pers} = 3], \\ \texttt{bark} &\mapsto \mathsf{V}[\mathsf{agr} = \mathsf{non3sg}] \quad \text{or} \quad \mathsf{N}[\mathsf{num} = \mathsf{sg}]. \end{aligned}$$

The parser first tries the verbal reading of `bark`. It builds an NP from `the dog`; that NP inherits singular third-person agreement from the noun. It builds a VP from `bark`; the base present form requires a non-third-person-singular subject. The sentence rule then demands

$$\mathsf{NP.agr} = \mathsf{VP.agr}.$$

For this input, that reduces to the incompatible unification

$$[\mathsf{num} : \mathsf{sg},\ \mathsf{pers} : 3] \sqcup [\mathsf{num} : \mathsf{pl} \text{ or non-3sg}] = \top.$$

The noun reading of `bark` leaves the parser with a sequence like Det–N–N, which no sentence rule licenses. Therefore strict parsing fails. But if the subject–verb agreement constraint is relaxed, the verbal parse completes. The engine can report:

```
the dog bark
        ^^^^


Verdict:   ungrammatical
Violation: subject-verb agreement
Rule:      S -> NP VP with <NP agr> = <VP agr>
Fix:       "the dog barks" or "the dogs bark"
```

The explanation is not a separate hand-written special case. It is the failed unification, viewed from the user's side.

14 Auxiliaries, clitics, and head projection

English auxiliaries show why morphology and syntax cannot be cleanly separated. In `I don't bark`, the token `n't` is not punctuation; it is part of the auxiliary structure. In `he doesn't bark`, subject agreement is carried by `does`, not by the lexical verb. In `he'll bark`, the modal `'ll` does not require the following verb to agree with the subject, but it does require that verb to appear in its base form.

The grammar therefore gives auxiliaries their own projection, AuxP, rather than treating them as ordinary lexical verbs. This has two advantages. First, auxiliary-specific constraints can be stated locally. Second, the parser does not accidentally merge the feature environment of an auxiliary with the feature environment of its complement verb.

The relevant distinction is not only agreement but *verb form*. English auxiliaries select particular forms of their complements: modals and do-support select a bare verb, perfect `have` selects a past participle, and progressive `be` selects a present participle. The engine represents this selection with a feature

$$\text{VFORM} \in \{\text{BARE},\ \text{FIN},\ \text{PASTP},\ \text{PRESP}\}.$$

Here bare covers base forms such as `bark`; FIN covers finite present and past forms; PASTP covers past participles; and PRP covers present participles such as `barking`.

The value FIN is intentionally coarse. English finite verbs still vary internally: present-tense verbs show agreement distinctions, and past-tense verbs usually do not. Those facts are handled by agreement features and by the lexical analyses emitted by morphology. For auxiliary selection, however, the main question is simply whether a verb can head a finite clause. Collapsing the finite forms into one value lets a main-clause rule require finiteness with a single equation:

```
<VP VFORM> = FIN.
```

English also contains substantial syncretism: one surface form may realize multiple grammatical forms. The engine handles this by ambiguity rather than by special cases. For example, `bark` may be analyzed both as a bare form and as a finite present form; `barked` may be analyzed both as a finite past form and as a past participle. The parser keeps all analyses whose features are compatible with the surrounding syntax and discards the rest.

Auxiliary selection is then just another feature equation:

```
AuxP -> Aux VP
    <VP vform> = <Aux req>   ! "wrong verb form after the auxiliary" fix: agree-verb
```

The auxiliary carries the verb form it requires in its `req` feature. Modals and do-support set `req = bare`; perfect `have` sets `req = pastp`; progressive `be` sets `req = prp`. The rule does not need separate procedural checks for each auxiliary. It only unifies the complement's `vform` with the auxiliary's requirement.

This rejects errors such as `I'll barked`. The clitic `'ll` is a modal auxiliary, so it requires a bare complement. The form `barked` can be finite past or past participle, but it is not bare, so the equation

```
<VP vform> = <Aux req>
```

fails. Because the diagnostic is attached to that equation, the report can name the error as a wrong verb form after the auxiliary and suggest the compatible base form: `I'll bark`.

15 Nominal and adjectival structure

The early examples in the engine focus on subject–verb agreement because it is the smallest diagnostic that demonstrates parsing plus unification. Expanding coverage quickly makes noun phrases more important. English noun phrases contain determiners, pronouns, adjectives, degree morphology, and stacked modifiers:

`the big old dog`, `the bigger dog`, `he`, `the dog`.

The point of adding these constructions is not to introduce a new mechanism, but to test whether the same grammar architecture scales to more internal structure.

15.1 Adjectives and stacking

An attributive adjective modifies an NP-internal nominal projection without changing the head noun's agreement features. This means adjective stacking can be handled by recursive phrase structure while head features continue to percolate from the noun.

```
Nom -> Adj Nom
    <Nom agr> = <Nom.1 agr>
```

15.2 Degree as a morphological feature

Comparative and superlative adjectives are still adjectives. The syntax does not need a separate category for `bigger` or `biggest`; morphology adds a degree feature:

$$\mathsf{degree} \in \{\mathsf{pos},\ \mathsf{comp},\ \mathsf{sup}\}.$$

Regular adjectives use a productive paradigm:

$$\texttt{big} \mapsto \texttt{bigger}, \texttt{biggest}, \qquad \texttt{happy} \mapsto \texttt{happier}, \texttt{happiest}.$$

Suppletive adjectives such as `good`, `better`, and `best` are lexical entries sharing a lemma rather than outputs of a spelling rule.

15.3 Pronouns as complete noun phrases

Pronouns differ from ordinary nouns because they can project a complete noun phrase without a determiner. They also carry case directly:

$$\texttt{he} : \mathsf{Pron}[\mathsf{case} = \mathsf{nom}], \qquad \texttt{him} : \mathsf{Pron}[\mathsf{case} = \mathsf{acc}].$$

This lets the same VP-object rule reject `the cat chased I` and accept `the cat chased me`.

16 The grammar language

The engine reads grammars from plain-text `.gram` files. The format is a small declarative language: lexicon entries define words, phrase rules define structure, feature equations define constraints, and directives configure morphology and modular loading.

Form	Meaning
`word : CAT <f>=v`	Lexicon entry with category and feature value.
`A -> B C`	Phrase-structure rule.
`<B f> = <C f>`	Feature equation attached to a rule.
`! "message" fix: ...`	Diagnostic attached to a constraint.
`%mal ... *ERR ... *FIX ...`	Targeted error pattern.
`%feature num : sg | pl`	Declares legal values for a feature.
`%class Vowel : a | e | i | o | u`	Defines a symbol class for morphology rules.
`%rule e:0 => Cons _ e d`	Morphophonemic rewrite rule.
`%include`, `%import`	File inclusion and TSV lexicon import.

Table 4: Major forms in the grammar language.

The `%feature` directive is a load-time type discipline. If a grammar declares

```
%feature num : sg | pl
```

then a typo such as `<num>=sgular` is rejected when the grammar loads rather than turning into a silent feature value that never unifies. Open features such as lemmas can be declared with `*`.

The `%include` and `%import` directives let `en.gram` become a manifest rather than a single monolithic file. A typical layout separates closed-class words, clitics, morphology, syntax, and imported open-class lexicons. For the browser build, the grammar can be flattened at build time so runtime analysis stays filesystem-free.

17 Trace mode and explainability

A useful engine needs to explain itself to two audiences: the user and the author. User-facing reports describe the violated grammatical rule. Trace mode exposes the internal computation that led there.

A trace records per-token morphology and the Earley chart operations: predictions, scans, completions, scan misses, and feature clashes. For an input such as `the dog bark`, a trace can show

that `bark` was considered as both noun and verb, that the noun path died structurally, and that the verb path reached a subject–verb agreement clash.

This is the same principle at two scales. The user sees a grammatical explanation; the developer sees the parser state that produced it.

18 Performance and accidental complexity

The formal architecture of the engine is designed to keep ordinary analysis cheap. Morphology is finite-state, parsing is polynomial, and feature constraints are checked locally as rules are applied. In principle, then, a larger lexicon should not make every part of the system slower. Adding more words should increase the size of the morphology, but it should not make the parser repeatedly search the entire vocabulary.

The heavy benchmark was built to test exactly that assumption. It imported a 10k-word noun lexicon and measured four kinds of input: ordinary grammatical sentences, direct mal-rule matches, agreement errors diagnosed by constraint relaxation, and missing-determiner errors diagnosed by repair generation.

Input class	n	p50	p95	max
grammatical	42	4.64 ms	6.15 ms	9.92 ms
mal-rule	14	8.94 ms	11.78 ms	11.78 ms
relaxed S–V agreement	14	44.29 ms	49.27 ms	49.27 ms
relaxed missing determiner	14	75.07 ms	90.66 ms	90.66 ms

Table 5: Analyze-time before indexing on a 10k-word imported noun lexicon.

The result separated the ordinary path from the diagnostic path. Grammatical inputs and direct mal-rule matches stayed in the low-millisecond range. Relaxed diagnostic recovery was much slower. That difference was the clue: the parser itself was not the main bottleneck. The slowdown appeared when the engine began generating candidate repairs and re-analyzing them.

The cause was an accidental scaling cliff in lexical decoding. For each tag sequence, the morphology layer scanned every root in the lexicon to find stems compatible with the surface word. This made morphological analysis

$$O(R)$$

per word, where R is the number of roots in the lexicon. On a small grammar, that cost was invisible. Inside diagnostic recovery, however, it was multiplied: each generated repair candidate triggered another analysis, and each analysis rescanned the full 10k-root lexicon.

This was not a flaw in the linguistic model. It was a local implementation choice whose asymptotic cost was hidden until the lexicon became large enough. The fix was to index roots by surface prefix, or equivalently to use the trie structure that already stores them. Candidate lookup then changes from a full scan over all roots to a walk over the possible prefixes of the input word.

The indexed walk preserves the old semantics. Overlapping stems such as `do` and `dog` are both found, each at its own prefix length. The prefix is built by concatenating emitted tape symbols rather than by indexing directly into the string, so multi-code-unit characters remain consistent with the representation used by the trie. The index is memoised on the parsed morphology object,

which is immutable after grammar loading. The $O(R)$ construction cost is therefore paid once per grammar rather than once per token.

Indexing the roots closed the lexicon-dependent part of the cliff. Indexing the repair candidates then reduced the remaining recovery overhead. On the original heavy benchmark, overall p95 fell from 77 ms to 34 ms after root indexing, and then to 16.7 ms after fix indexing.

The grammar has since grown beyond subject–verb agreement to include adjectives, degree morphology, coordination, adverbs, pronouns, and the auxiliary system, so the benchmark was re-run on the current fragment. Two things hold. First, the cliff stays closed: adding a 10k-word lexicon leaves the relaxed numbers essentially unchanged, so analysis is no longer dependent on lexicon size. Second, the remaining cost of diagnostic recovery comes from fix-candidate generation and re-parsing over the grammar, not from vocabulary lookup.

Input class	p50	p95	max
grammatical	4.1 ms	5.1 ms	5.3 ms
grammatical (adjective)	4.4 ms	7.8 ms	7.8 ms
grammatical (coordination)	4.4 ms	4.8 ms	4.8 ms
mal-rule	6.0 ms	7.4 ms	7.4 ms
relaxed S–V agreement	27.1 ms	39.2 ms	39.2 ms
relaxed missing determiner	34.2 ms	90.9 ms	90.9 ms
relaxed coordination	50.7 ms	88.0 ms	88.0 ms

Table 6: Analyze-time on the current grammar with a 10k-word imported lexicon. Grammatical and mal-rule inputs remain cheap. Diagnostic recovery is costlier because it generates and re-parses repair candidates; its cost now scales with the number of triggered repair strategies rather than with the lexicon size.

The broader lesson is that accidental complexity often enters through the edges of an otherwise sound architecture. The formal design made morphology regular, parsing polynomial, and diagnostics local, but one lookup routine still made recovery scale with the full lexicon. Precomputing an index keyed by the operation actually being performed removed that accidental complexity without changing the linguistic model.

19 Scope, limitations, and readiness for expansion

The current system is a small English fragment, not a wide-coverage grammar. It is intentionally conservative: it reports only errors it can localize through declared constraints, relaxed parses, or mal-rules. This gives up coverage in exchange for trustworthiness. A missing parse is not treated as proof of a mistake unless the engine can point to the rule that failed.

The last architecture-level blocker before grammar growth was constituent addressing. In the early format, a path such as `<NP num>` named a constituent by category symbol. That works when a production contains only one noun phrase, but it fails for coordination and ditransitives:

```
NP -> NP Conj NP,     VP -> V NP NP.
```

Repeated categories now require aliases, and ambiguous bare names fail at load time. This means the grammar can grow without later rewriting hundreds of rules to distinguish same-category daughters.

The first expansions beyond subject–verb agreement were chosen to stress different parts of the pipeline. Pronouns test case features and noun-phrase projection. Adjectives test recursive modifier structure and head-feature percolation. Comparative and superlative forms test productive morphology, spelling rules, and suppletion. Adverbs test modifier attachment outside the noun phrase. Coordination tests repeated-category addressing and agreement across parallel structures.

Together, these additions check that the engine is not a collection of isolated grammar checks. Each new construction enters through the same pipeline: morphology proposes analyses, the parser builds structure, feature equations enforce constraints, and diagnostics are reported only when a named failure can be localized.

The next expansion risk is therefore no longer the basic representation, but ambiguity management. As the grammar gains more modifiers, attachment sites, and lexical categories, the parser will naturally produce more competing analyses. The engine must continue to preserve useful ambiguity while suppressing spurious diagnostics. That is the main correctness pressure on future lexicon and rule growth.

20 The grammar file format

The previous sections describe the engine abstractly: morphology is finite-state, syntax is parsed over a context-free backbone, and grammatical constraints are checked by unifying feature structures. The `.gram` file is where those ideas become executable data. It is the engine's source language.

This is the compiler analogy at its most literal. A `.gram` file is not a configuration file in the weak sense of toggling options; it is a small declarative program. Its lexicon entries introduce symbols, its phrase-structure rules describe admissible local trees, its equations impose feature constraints, and its diagnostic annotations say which failures are meaningful enough to report. Loading a grammar therefore resembles compiling a source file: includes and imports are resolved, declarations are checked, morphology is compiled, and ill-typed feature values are rejected before analysis begins.

The format is PATR-II in spirit, following the unification-grammar tradition, but extended with the pieces this engine needs for grammatical diagnosis: feature declarations, morphology directives, diagnostic annotations, and mal-rules for known wrong patterns.

There are four main families of statements:

1. lexicon entries, which attach categories and features to surface forms;
2. phrase-structure rules, which build constituents and impose equations;
3. mal-rules, which recognize known ungrammatical patterns directly;
4. directives, which configure the grammar loader and morphology compiler.

Whitespace separates tokens but is otherwise insignificant. A `#` character begins a comment running to the end of the line, and blank lines are ignored. The grammar of the format is as follows, where `::=` defines a nonterminal, `|` marks alternatives, `{}` means zero or more repetitions, `[]` marks an optional part, and `EOL` is the end of a line.

```
grammar     ::= { lexentry | rule | malrule | directive }

lexentry    ::= word ":" cat { featpath "=" value } EOL
```

```
rule        ::= production { equation }
production  ::= cnst "->" cnst { cnst } EOL
cnst        ::= [ alias ":" ] sym
                # a constituent: a category, optionally named by an alias

equation    ::= rulepath "=" ( rulepath | value ) [ diag ] EOL
diag        ::= "!" string [ "fix:" strategy { "|" strategy } ]

malrule     ::= "%mal" production "*ERR" string [ "*FIX" text ]

directive   ::= feature | class | replrule | include | import
feature     ::= "%feature" feature ":" ( value { "|" value } | "*" ) EOL
class       ::= "%class" name ":" sym { "|" sym } EOL
replrule    ::= "%rule" pat ":" pat [ "=>" { ctxsym } "_" { ctxsym } ] EOL
include     ::= "%include" path EOL
import      ::= "%import"  path EOL

ctxsym      ::= sym | name

featpath    ::= "<" feature { feature } ">"
rulepath    ::= "<" name { feature } ">"

word        ::= token
cat, sym    ::= identifier
alias, name ::= identifier
feature     ::= identifier
value       ::= identifier
strategy    ::= identifier
string      ::= '"' { char } '"'
```

The syntax is intentionally small, but several semantic conventions are doing important work.

First, `=` is unification, not assignment. An equation such as `<NP agr> = <VP agr>` does not copy a value from one side to the other. It declares that the two feature structures must be compatible. When a rule is applied, all of its equations are unified together. If every merge succeeds, the rule builds a constituent with the resulting information. If any merge conflicts, the rule application fails and that parse branch dies. This is the same operation described in Section 5, now exposed as grammar syntax.

Second, paths name pieces of feature structure. In a lexicon entry, the constituent is implicit: the path refers to the entry's own feature structure.

```
dog : N <num>=sg <pers>=3
```

This says that `dog` is a noun whose number is singular and whose person is third. In a phrase-structure rule, the first name inside the angle brackets selects one of the constituents in the production, and the remaining names walk into that constituent's feature structure.

```
S -> NP VP
  <NP agr> = <VP agr> ! "subject-verb agreement" fix: agree-verb
```

Here the rule builds a sentence from a noun phrase and a verb phrase, but only when their agreement features unify. The diagnostic annotation says that if relaxing exactly this equation rescues the parse, the failure should be reported as a subject-verb agreement error.

Third, omitted information is not false information. A missing feature is unconstrained and unifies with anything. This matters because many grammatical facts are best expressed by silence. For example, if a past-tense English verb does not participate in present-tense agreement, the grammar does not need to write a special value meaning "irrelevant"; it can simply omit the agreement feature.

Fourth, a `%mal` rule recognizes an ungrammatical pattern directly. A diagnostic annotation on an ordinary rule says, "this constraint may explain a failure." A mal-rule says, "this wrong structure is known and should always be reported when matched." The two mechanisms complement each other: relaxation can discover local constraint failures, while mal-rules cover recurrent errors that are easier to recognize as patterns.

Directive lines are discharged before ordinary parsing begins. The loader expands `%include` and `%import`, checks `%feature` declarations, records `%class` symbol sets for morphology rules, and adds `%rule` declarations to the morphophonemic rewrite cascade. This front-end pass is why many grammar mistakes are caught at load time rather than turning into silent parse failures later.

20.1 Constituent addressing

A subtle issue appears once a production contains two constituents of the same category. In the early grammar format, a path such as `<NP num>` named a constituent by its category symbol. That works for a rule like

```
S -> NP VP
```

because there is only one noun phrase. It fails for rules such as coordination or ditransitives:

```
NP -> NP Conj NP
VP -> V NP NP
```

In those productions, the category name `NP` is no longer enough to identify a unique constituent. A symbol-keyed environment silently conflates the two daughters, so one noun phrase can overwrite the other.

The current format solves this by giving every constituent occurrence a distinct identity. A production may also assign aliases to constituents, and equations may refer to those aliases:

```
NP -> left:NP Conj right:NP
  <left num> = <right num>

VP -> give:V obj1:NP obj2:NP
  <obj1 case> = acc
  <obj2 case> = acc
```

Name resolution happens at load time. A name in a rule path is resolved in this order:

1. a declared alias, if one exists;
2. the left-hand-side category, which always names the mother;

3. a daughter category, but only if that category occurs exactly once.

If a category occurs more than once among the daughters, it must be aliased. Using the bare category name is a load-time error. This makes ambiguous grammar rules fail early, before they can produce incorrect parse environments.

This addressing scheme is separate from head-feature inheritance. When the left-hand-side category occurs exactly once among the daughters, the mother may default to that daughter's structure. Thus a rule such as

```
NP -> NP PP
```

can still percolate the head noun phrase without writing explicit equations. That inheritance convention is not an addressing rule; it is a linguistic default. This distinction is why auxiliary projections remain useful. A modal auxiliary should not inherit the agreement requirements of its complement verb, so the grammar gives it a distinct projection such as `AuxP`.

21 Conclusion

gram.en treats natural-language grammar checking as a compiler-front-end problem. The formal language theory explains the computational boundaries: regular methods are ideal for morphology, context-free parsing is the practical syntactic baseline, and unrestricted grammar is too powerful to decide. Feature structures and unification add the local constraints needed for agreement, case, and selection. Mal-rules and constraint relaxation turn failures into named, localized diagnostics while respecting the coverage trap.

The resulting engine is deliberately limited, but its limitations are part of the design. It does not try to model all of natural language. It asks how much useful grammatical diagnosis can be obtained from explicit, inspectable rules. Within that scope, the verdict and the explanation are not separate products. They are the same computation viewed from two sides.

A References

[1] N. Chomsky. *Three models for the description of language.* IRE Transactions on Information Theory, 1956.

[2] S. M. Shieber. *Evidence against the context-freeness of natural language.* Linguistics and Philosophy, 1985.

[3] A. K. Joshi. *Tree adjoining grammars: How much context-sensitivity is required to provide reasonable structural descriptions?* In *Natural Language Parsing*, 1985.

[4] K. Vijay-Shanker and D. J. Weir. *The equivalence of four extensions of context-free grammars.* Mathematical Systems Theory, 1994.

[5] G. K. Pullum and B. C. Scholz. *On the distinction between model-theoretic and generative-enumerative syntactic frameworks.* LACL, 2001.

[6] R. Kasper and W. Rounds. *A logical semantics for feature structures.* ACL, 1986.

[7] D. Jurafsky and J. H. Martin. *Speech and Language Processing*, 3rd edition draft.

[8] M. Sipser. *Introduction to the Theory of Computation.*

[9] S. M. Shieber. *An Introduction to Unification-Based Approaches to Grammar*. CSLI, 1986.

[10] F. C. N. Pereira and S. M. Shieber. *Prolog and Natural Language Analysis*. CSLI, 1987.

[11] R. M. Kaplan and M. Kay. *Regular models of phonological rule systems*. Computational Linguistics, 1994.

[12] R. Kaplan and M. Kay; K. Koskenniemi. Foundational work on finite-state and two-level morphology.